

PARNAS – “CREDIFAB”

VISÃO DO PRODUTO E DO PROJETO

Versão 2.3

17 de junho de 2026

Tabela 1 – Integrantes do Grupo

Matrícula	Nome	Papéis	Pontos
241025499	Alan Semil dos Santos Vieira	Desenvolvedor Backend	8,34
232024509	Anna Júlia Aparecida Silva Primo	Desenvolvedora Frontend	8,34
242015782	Cibelle de Assis Silva	Desenvolvedora Frontend	8,34
251019771	Daniel Filipe Borges de Oliveira	Analista de Qualidade e Testador de Software	8,34
251008114	Eduardo Jesus Dal Pizzol	Líder do Grupo; Arquiteto/Analista de Banco de Dados e Domínio	8,34
251009185	Gabriel Melo Rodrigues Cardone	Desenvolvedor Backend	8,34
251010186	Igor Dantas Araújo	Desenvolvedor Backend	8,34
242028708	João Marcos Santos e Carvalho	Desenvolvedor Frontend	8,34
251023184	João Pedro da Nóbrega Souza Cordeiro	Desenvolvedor Backend	8,34
242015871	Júlia Amanda Silva Lima	Desenvolvedora Frontend	8,34
242028842	Maria Eduarda de Oliveira Gomes	Arquiteta/Analista de Banco de Dados e Domínio	8,34
251013660	Matheus Moretti Soares	Analista de Qualidade e Testador de Software	8,34

Fonte: elaborado pelo autor (2026).

**Os pontos de contribuição foram distribuídos igualmente entre todos os integrantes, pois todos foram autores deste documento.*

Tabela 2 – Histórico de Revisões

Data	Versão	Descrição	Autor
06/05/2026	0.1	Correções de especificações incorretas e/ou incoerentes ao longo do documento, tal qual as tecnologias utilizadas e declarações de métricas.	Todos os integrantes do grupo.
07/05/2026	1.0	Inclusão da tabela de Roteiro de Testes e padronização das inclusões de tabelas ao longo do texto. Assim, houve a finalização da primeira versão de entrega do documento de visão.	Daniel e Matheus.
27/05/2026	2.0	Reestruturação do documento: correção de tabelas, padronização de informações e esclarecimento do escopo do produto. Repaginação nas seções de GQM e Testes.	Daniel.
30/05/2026	2.1	Reajuste das funcionalidades.	Todos os integrantes do grupo.
30/05/2026	2.2	Reajuste no Roteiro de Testes.	Daniel, João e Matheus.
17/06/2026	2.3	Reajuste no Roteiro de Testes e Métricas.	Daniel e Matheus.

Fonte: elaborado pelo autor (2026).

Sumário

Integrantes do Grupo	1
Histórico de Revisões	2
1 VISÃO GERAL DO PRODUTO	4
1.1 Problema	4
1.2 Declaração de Posição do Produto	7
1.3 Objetivos do Produto	7
1.4 Tecnologias a Serem Utilizadas	8
2 VISÃO GERAL DO PROJETO	8
2.1 Ciclo de vida do projeto de desenvolvimento de software	8
2.1.1 Ciclo de Vida do Projeto	8
2.2 Organização do Projeto	9
2.3 Planejamento das Fases e/ou Iterações do Projeto	10
2.4 Matriz de Comunicação	12
2.5 Gerenciamento de Riscos	13
2.6 Critérios de Replanejamento	14
3 PROCESSO DE DESENVOLVIMENTO DE SOFTWARE	14
4 DECLARAÇÃO DE ESCOPO DO PROJETO	15
4.1 Backlog do produto	15
4.2 Perfis	16
4.3 Cenários	16
4.4 Tabela de Backlog do produto	17
5 MÉTRICAS E MEDIÇÕES	18
5.1 GQM de medições	18
5.2 Consolidação das Métricas por Sprint	21
6 TESTES DE SOFTWARE	22
6.1 Estratégia de Testes	22
6.2 Roteiro de Teste	25
7 REFERÊNCIAS BIBLIOGRÁFICAS	28
Referências Bibliográficas	28

1 VISÃO GERAL DO PRODUTO

1.1 Problema

Contexto: Dentro da temática da ODS 9 – construir infraestruturas resilientes, promover industrialização inclusiva e sustentável e fomentar inovação –, em vista da Meta 9.3, aumentar o acesso das pequenas indústrias e outras empresas, particularmente em países em desenvolvimento, aos serviços financeiros, incluindo crédito acessível e sua integração em cadeias de valor e mercados.

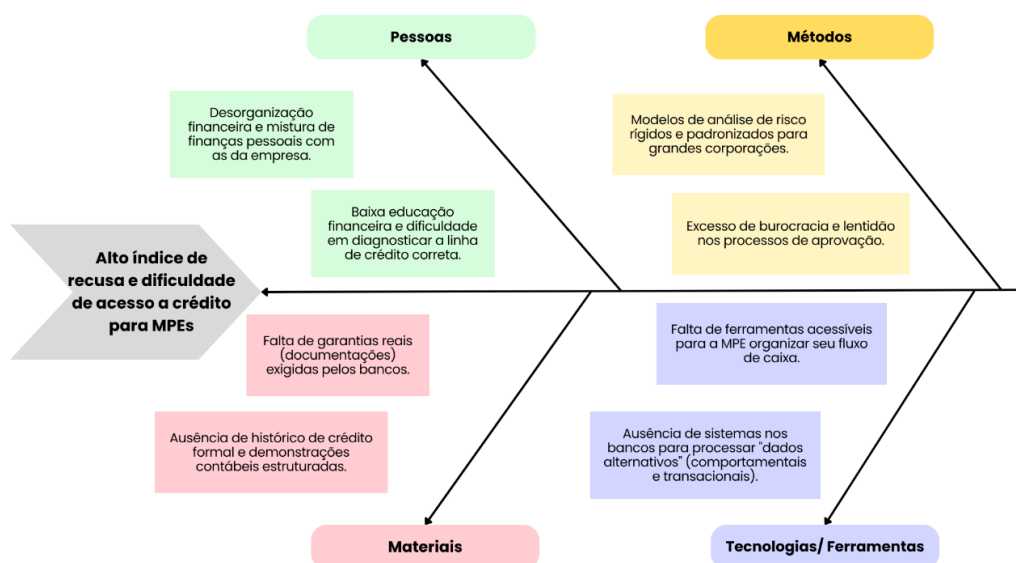
A exclusão financeira crônica dos pequenos negócios é um desafio estrutural. No Brasil, as micro e pequenas empresas (MPEs) representam cerca de 99% dos estabelecimentos e respondem por mais da metade dos empregos formais (REDAÇÃO HOMEWORK, 2025). No entanto, historicamente, essas empresas absorvem uma parcela desproporcionalmente baixa do crédito concedido pelo sistema financeiro, figurando na faixa de cerca de 11% do volume total (SEBRAE, 2025). Essa disparidade ilustra o gargalo exato que a ODS 9.3 busca resolver.

O cerne da recusa de crédito pelas instituições financeiras reside na alta assimetria de informações entre quem pede o empréstimo e quem o concede (SEBRAE, 2025). Bancos tradicionais baseiam suas análises de risco em demonstrações contábeis rigorosas, contudo, os balanços das micro e pequenas empresas muitas vezes não expressam a realidade financeira devido à informalidade e à comum mistura do patrimônio dos donos como pessoa física com o da empresa (BUENO, 2003; PREISLER, 2003). Além disso, muitos empreendedores informais sofrem com a desorganização financeira e não possuem documentação estruturada sobre a sua empresa para solicitar o crédito adequadamente como pessoa jurídica.

O modelo tradicional de concessão é lento e esbarra no excesso de exigências documentais, o que figura entre os principais impedimentos para a aprovação (SEBRAE, 2025). Paralelo à burocracia dos bancos, o próprio empreendedor tem grande dificuldade em realizar o diagnóstico correto de sua necessidade financeira, por exemplo, não sabendo identificar qual linha de crédito faz sentido para a finalidade do seu negócio e tendo dificuldades para comparar as diferentes opções existentes no mercado.

Problema encontrado: Micro e pequenas empresas e indústrias têm grande dificuldade de acesso e concessão de crédito bancário, além disso se deparam com taxas injustas, porque não sabem calcular os seus custos ou apresentar a sua saúde financeira de forma estruturada para as instituições financeiras.

Figura 1: Diagrama de Ishikawa.



Fonte: Elaborada pelos autores (2026).

A partir do Diagrama de Ishikawa apresentado na Figura 1 e da contextualização do problema, fica evidente que o alto índice de recusa de crédito ocorre devido a falhas sistêmicas e dificuldades encontradas em múltiplas frentes.

Nas categorias de **Pessoas** e **Materiais**, observamos que o pequeno empresário busca crédito de forma reativa, muitas vezes quando já está endividado, e não possui os materiais necessários (histórico e documentação) exigidos pelo modelo tradicional (GUIMARÃES, 2021). Há uma grave dificuldade de diagnosticar a real necessidade do negócio, o que resulta na escolha da linha de crédito errada e agrava o risco (BRASIL, [s.d.]; REDAÇÃO HOMEWORK, 2025).

Nas categorias de **Métodos** e **Tecnologia**, o diagrama expõe que os bancos utilizam métricas antiquadas, focadas em exigir garantias reais robustas que a grande maioria dos pequenos negócios não possui. Além disso, os processos são muito lentos e geram a chamada assimetria de informação: instituição financeira não consegue visualizar a saúde real da empresa em tempo real, enquanto o empresário não compreende as exigências do banco (SEBRAE, 2025).

Identificação e justificativa da nossa solução de software:

A solução do software proposta consiste no desenvolvimento de uma aplicação web voltada à governança financeira e preparação para o crédito. Nosso produto se diferencia ao focar na estruturação de dados transacionais e documentais sob a ótica das exigências das instituições financeiras, atuando como um facilitador tecnológico para o cumprimento da Meta 9.3 da ODS 9.

A justificativa para o desenvolvimento desta solução vem da necessidade de resolver o desalinhamento entre a realidade operacional do pequeno empresário e os modelos

de análise de risco bancário. A contribuição esperada dessa aplicação se estrutura nos seguintes pontos:

- **Padronização de dados financeiros:** facilitar a geração de balanços e fluxos de caixa, fornecendo uma “vitrine” de saúde financeira que reduz a assimetria de informações. Espera-se que a transparência dos dados permita que o tomador de crédito seja avaliado por sua capacidade real de pagamento.
- **Unificação e Centralização Documental:** criação de um repositório unificado que centraliza documentos jurídicos, fiscais e contábeis essenciais. Ao organizar essa documentação de forma padronizada, o software elimina a fragmentação de informações que hoje figura como um dos principais impedimentos para a aprovação de crédito. Espera-se que a unificação reduza drasticamente o retrabalho e o tempo de resposta durante o processo de solicitação, além de facilitar a organização do gestor.
- **Apoio à Decisão e Diagnóstico de Crédito:** propor um diagnóstico preciso da necessidade de capital, auxiliando na identificação da linha de crédito mais adequada para a finalidade do negócio. Ao comparar taxas e modalidades, o sistema visa evitar que o empresário recorra ao crédito de pessoa física, cujas taxas superam 133% ao ano, promovendo o uso de crédito jurídico mais acessível, em torno de 23% ao ano.

Em suma, a aplicação contribuirá para a solução do problema ao elevar a maturidade administrativa e a visibilidade das micro e pequenas indústrias perante o sistema financeiro. O resultado esperado é a facilitação do processo de concessão de crédito e a democratização do acesso aos serviços financeiros, garantindo que a infraestrutura e a inovação propostas pela ODS 9 sejam viabilizadas por meio de uma gestão financeira resiliente, unificada e transparente.

1.2 Declaração de Posição do Produto

Tabela 3 – Posicionamento do CREDIFAB. Fonte: elaborado pelo autor (2026).

Para:	Proprietários, gestores financeiros e administradores de micro e pequenas empresas e indústrias.
Necessidade:	Estruturar dados financeiros e documentais para aumentar a capacidade de obtenção de crédito e melhorar a gestão empresarial.
O CREDIFAB:	É uma aplicação web de gestão financeira e preparação para acesso ao crédito.
Que:	Organiza finanças, gera relatórios confiáveis e orienta a melhor escolha de crédito para o negócio.
Ao contrário:	Métodos improvisados, sistemas genéricos e decisões financeiras tomadas sem dados concretos.
Nosso produto:	Une organização financeira, inteligência de crédito e simplicidade operacional em uma plataforma feita para pequenas empresas brasileiras.

1.3 Objetivos do Produto

Objetivo Geral: Desenvolver uma aplicação web denominada CREDIFAB, voltada às micro e pequenas empresas e indústrias, capaz de organizar informações financeiras e documentais, apoiar a tomada de decisão e facilitar o acesso ao crédito em condições mais adequadas, contribuindo para o cumprimento da meta 9.3 da ODS 9.

Objetivos Específicos:

- Centralizar dados financeiros da empresa, como receitas, despesas e fluxo de caixa.
- Permitir o armazenamento e organização de documentos empresariais relevantes para solicitação de crédito.
- Gerar relatórios financeiros simples e compreensíveis para gestores e instituições financeiras.
- Auxiliar o usuário na identificação da real necessidade de capital para o negócio.
- Comparar modalidades de crédito, taxas e condições disponíveis no mercado.
- Simular financiamentos e impactos das parcelas no caixa da empresa.

1.4 Tecnologias a Serem Utilizadas

A princípio, o projeto fará uso das seguintes tecnologias e ferramentas:

- **Backend:** Python, Flask;
- **Frontend/UI:** React, Vite;
- **Banco de Dados:** SQLAlchemy, PostgreSQL;
- **Deploy e Hospedagem:** Render (Backend) e GitHub Pages (Frontend);
- **Testes:** Pytest, pytest-mock, pytest-cov, Vitest, React Testing Library, Playwright e Locust;
- **Documentação:** MKDocs e Overleaf.
- **Comunicação:** Discord e WhatsApp.

2 VISÃO GERAL DO PROJETO

2.1 Ciclo de vida do projeto de desenvolvimento de software

2.1.1 Ciclo de Vida do Projeto

O projeto será desenvolvido utilizando uma metodologia ágil, mediante conceitos de Scrum e XP. Haverá desenvolvimento em pares organizado em sprints ao longo do semestre.

Cada sprint terá duração fixa de uma semana, com cerimônias de planejamento no início e revisão/retrospectiva ao fim. O trabalho em pares garante revisão contínua do código e distribuição do conhecimento entre os membros da equipe.

O ciclo se divide nas seguintes fases:

- **Sprint 1 – Definição do Produto:** Levantamento de requisitos, definição do escopo, criação do backlog inicial e identificação das necessidades de micro e pequenas empresas e indústrias em relação à gestão de investimentos e planejamento financeiro.
- **Sprint 2 – Protótipo e Planejamento Arquitetural:** Definição da versão mínima funcional do sistema junto da criação de um protótipo de baixa fidelidade. Definição da arquitetura e tecnologias adotadas.
- **Sprint 3 – Ambientação:** Organização do fluxo de trabalho, agendamento das reuniões, estruturação do repositório no GitHub e configuração inicial da pipeline de Integração Contínua (GitHub Actions).
- **Sprints 4 a 9 – Construção incremental:** Implementação progressiva das funcionalidades em código, com testes contínuos, distribuídas de forma homogênea (em média duas funcionalidades por Sprint).

- **Sprint final (10) – Estabilização e Entrega:** Consolidação do produto, testes E2E e de carga, correções de defeitos remanescentes e apresentação da solução completa.

2.2 Organização do Projeto

A equipe está organizada para garantir que todas as responsabilidades do Scrum sejam atendidas, sem hierarquia rígida, onde todos são responsáveis pelo sucesso do produto.

Tabela 4 – Papel dos integrantes.

Papel	Descrição	Participantes
Desenvolvedor Backend	Responsável pela arquitetura e implementação dos serviços da aplicação utilizando Python e Flask, incluindo a definição dos endpoints da API REST e regras de negócio, com deploy via Render.	Alan Semil dos Santos Vieira, João Pedro da Nóbrega Souza, Gabriel Melo Rodrigues Cardone, Igor Dantas Araújo
Desenvolvedor Frontend	Responsável pela arquitetura de componentes e implementação da interface utilizando React; definição de padrões de estado, integração com a API e hospedagem via GitHub Pages.	Anna Júlia Aparecida Silva Primo, Júlia Amanda Silva Lima, João Marcos Santos e Carvalho, Cibelle de Assis Silva
Arquiteto/Analista de Banco de Dados e Domínio	Responsável pela modelagem conceitual e lógica do banco de dados, definição das entidades, relacionamentos e regras de persistência utilizando SQLAlchemy e PostgreSQL.	Eduardo Jesus Dal Pizzol, Maria Eduarda de Oliveira Gomes
Analista de Qualidade e Testador de Software	Responsável por garantir a qualidade do produto, definir a estratégia de testes, elaborar planos e casos de teste e revisar artefatos produzidos pelo time.	Daniel Filipe Borges de Oliveira, Matheus Moretti Soares

Fonte: elaborado pelo autor (2026).

2.3 Planejamento das Fases e/ou Iterações do Projeto

A distribuição das entregas foi revisada nesta versão para equilibrar a carga entre as Sprints de construção, alocando em média duas funcionalidades por iteração e alinhando cada Sprint a um cenário funcional (Tabela 8).

Tabela 5 – Sprints.

Sprint	Início	Fim	Entregas
Sprint 1	26/04	02/05	Documento de Visão do Produto.
Sprint 2	03/05	09/05	Documento de Arquitetura.
Sprint 3	10/05	16/05	Ambientação: estruturação do repositório.
Sprint 4	17/05	23/05	Funcionalidade A (parcial).
Sprint 5	24/05	30/05	Funcionalidades A e B.
Sprint 6	31/05	06/06	Funcionalidades C e D.
Sprint 7	07/06	13/06	Funcionalidades E e J.
Sprint 8	14/06	20/06	Funcionalidades G e F.
Sprint 9	21/06	27/06	Funcionalidades H e I.
Sprint 10	28/06	04/07	Estabilização final: testes E2E (Playwright), teste de carga (Locust), correções e produto final.

Fonte: elaborado pelo autor (2026).

Funcionalidades

Funcionalidade A – Cadastro e Autenticação de Usuário/Empresa. Permite que representantes de microempresas criem uma conta na plataforma, realizem login com credenciais seguras e gerenciem o acesso ao sistema. A autenticação é baseada em tokens JWT, garantindo segurança nas sessões. Cada conta está vinculada a uma empresa, isolando os dados financeiros de diferentes organizações.

Funcionalidade B – Categorização Financeira. Permite classificar cada transação em categorias predefinidas ou personalizadas, como fornecedores, folha de pagamento, impostos, aluguel e outros. Facilita a identificação de onde os recursos da empresa estão sendo alocados e apoia decisões de corte ou redistribuição de custos.

Funcionalidade C – Cadastro de Transações Financeiras. Permite o registro manual de entradas e saídas financeiras da empresa, com informações como valor, data, tipo (receita ou despesa), categoria e descrição. É a funcionalidade central do sistema, sendo base para todas as análises e relatórios gerados pela plataforma.

Funcionalidade D – Histórico de Transações. Disponibiliza uma listagem completa de todas as movimentações financeiras registradas, com suporte a filtros por período, categoria, tipo e valor. Permite que o gestor consulte o histórico detalhado da empresa de forma rápida e rastreável.

Funcionalidade E – Dashboard Financeiro. Apresenta uma visão consolidada da situação financeira da empresa em tempo real, exibindo o saldo atual, total de receitas, total de despesas e evolução financeira do período selecionado. Os dados são apresentados por meio de gráficos e indicadores de fácil interpretação, mesmo para usuários sem formação financeira.

Funcionalidade F – Relatórios Financeiros. Gera resumos financeiros por período (mensal ou anual), apresentando receitas, despesas, saldo e distribuição por categoria em formato visual, exportável em PDF. Auxilia o gestor na análise do desempenho financeiro da empresa ao longo do tempo e na identificação de tendências.

Funcionalidade G – Centralização Documental. Permite que o gestor envie e organize documentos fiscais, contábeis e jurídicos da empresa em um repositório único. Centraliza informações historicamente fragmentadas e mantém a documentação padronizada para apresentação a instituições financeiras em processos de solicitação de crédito.

Funcionalidade H – Simulação de Crédito. Permite ao gestor simular operações de crédito a partir de parâmetros como valor solicitado, prazo, modalidade e taxa de juros. O sistema calcula o valor das parcelas (Tabela Price e SAC) e projeta o impacto no fluxo de caixa da empresa, apoiando a decisão sobre quanto e quando captar.

Funcionalidade I – Comparação de Modalidades de Crédito. Permite comparar diferentes modalidades de crédito lado a lado, considerando taxas, prazos e condições. Auxilia o pequeno empresário a escolher a linha de crédito mais adequada ao negócio, destacando a opção de menor custo total e alertando quando uma modalidade é de pessoa física, cujas taxas são significativamente mais altas que as de pessoa jurídica.

Funcionalidade J – Cadastro de Contas. Permite registrar e acompanhar contas a pagar e a receber da empresa, com valor, vencimento e situação (em aberto, paga ou recebida). Complementa o controle de fluxo de caixa ao dar visibilidade sobre compromissos financeiros futuros.

2.4 Matriz de Comunicação

Estratégias de Comunicação:

Tabela 6 – Matriz de Comunicação.

O Quê	Como	Quando	Responsável	Destinatário
Revisão & Planejamento de Sprint	Discord (online)	Sábados	Todos	Membros
Feedback Diário	WhatsApp	Todos os dias	Todos	Membros
Reunião Semanal Optativa	Discord (online)	Quartas-feiras	Todos	Membros
Comunicação Diária	WhatsApp & Discord	Todos os dias	Todos	Membros
Atas de Reunião	Overleaf	Quintas e Sábados	Daniel Filipe	Membros

Fonte: elaborado pelo autor (2026).

O controle de versão do projeto segue uma estratégia de branches estruturada para garantir a estabilidade do produto em todas as fases do desenvolvimento:

- **main** — branch de produção. Recebe merges apenas de código revisado, testado e aprovado pelo par de Qualidade. Representa sempre a versão estável e entregável do produto.
- **develop** — branch de integração contínua. Todo desenvolvimento é integrado aqui antes de ir para a main. Serve como ambiente de validação entre os pares.
- **feature/nome-da-funcionalidade** — branches temporárias criadas a partir da develop para o desenvolvimento de cada funcionalidade. Após revisão e aprovação via Pull Request, são mescladas de volta à develop e excluídas.
- **test/nome-da-funcionalidade-testada** — branches temporárias criadas a partir de branches de feature, para a realização dos devidos testes. Após a validação da feature, a sua branch de origem é mesclada à develop e a branch de teste é excluída.
- **fix/nome-do-bug** — branches temporárias para correção de bugs identificados durante os testes, seguindo o mesmo fluxo das branches de teste.

Todo merge na develop e na main requer aprovação via Pull Request, garantindo rastreabilidade e revisão entre os pares antes de qualquer integração.

2.5 Gerenciamento de Riscos

Foram identificados riscos que podem impactar o desenvolvimento do CREDIFAB.

- **Risco:** Curva de aprendizado das tecnologias (Flask/React).
 - *Mitigação:* Realizar workshops internos e utilizar o pair programming para nivelar o conhecimento técnico da equipe.
- **Risco:** Falta de disponibilidade de integrantes.
 - *Mitigação:* Manter a comunicação ativa via Discord e redistribuir tarefas imediatamente caso um par fique sobrecarregado ou ausente.
- **Risco:** Atraso na integração Backend e Frontend.
 - *Contingência:* Antecipar a definição de contratos de API (endpoints e formatos de dados).
- **Risco:** Inconsistência na modelagem do banco de dados.
 - *Mitigação:* Realizar revisões constantes do modelo lógico com os Analistas de Banco de Dados e validar a estrutura com o professor antes do início da implementação pesada.
- **Risco:** Desconformidade com os padrões de codificação (PEP8 e Docstrings).
 - *Mitigação:* Configurar linters no ambiente de desenvolvimento e realizar revisões de código (code reviews) antes de cada merge na branch principal.
- **Risco:** Conflito com o calendário acadêmico (provas e outros projetos).
 - *Mitigação:* Planejar as sprints com uma carga de trabalho reduzida em semanas críticas de avaliações, conforme o cronograma da disciplina.
- **Risco:** Falhas na segurança ou vazamento de dados simulados.
 - *Mitigação:* Implementar práticas básicas de segurança no Flask, como a gestão de variáveis de ambiente para chaves sensíveis, e evitar o uso de dados reais de empresas nos testes.

2.6 Critérios de Replanejamento

O replanejamento será acionado caso:

- Ocorra um atraso no cronograma de uma Sprint.
- Haja mudança crítica nos requisitos solicitados pelo “cliente”/professor/monitor.
- A saída de membros da equipe comprometa a capacidade produtiva.
- Identificação de dívida técnica acumulada que impeça novas funcionalidades.

Qualquer alteração nestes critérios resultará em uma nova versão deste documento.

3 PROCESSO DE DESENVOLVIMENTO DE SOFTWARE

O grupo realizará uma abordagem híbrida baseada nos frameworks ágeis Scrum e Extreme Programming (XP), com o intuito de garantir organização e qualidade contínua ao longo do projeto.

O Scrum será utilizado como estrutura de gerenciamento do projeto, organizando as atividades em Sprints ao longo do semestre. Cada Sprint terá duração fixa de 7 (sete) dias e será composta pela cerimônia de planejamento (Sprint Planning) e retrospectiva (Sprint Retrospective), além de uma reunião secundária no meio do período. Permitindo assim a inspeção e adaptação contínua do processo. Além disso, o grupo adotará a política de daily feedback, em que utilizamos um canal no WhatsApp para comunicar os outros membros sobre o que fizemos no dia em relação ao projeto. Essa abordagem iterativa e incremental possibilita entregas frequentes de valor e melhor controle sobre riscos e mudanças.

Complementarmente, o XP será aplicado nas práticas de desenvolvimento, com foco na qualidade do código e na eficiência do processo. Destaca-se o uso de desenvolvimento em pares (pair programming), que promove revisão contínua do código, compartilhamento de conhecimento e redução de erros. Além disso, serão adotadas práticas como testes automatizados, refatoração contínua e desenvolvimento incremental, contribuindo para a melhoria constante do sistema.

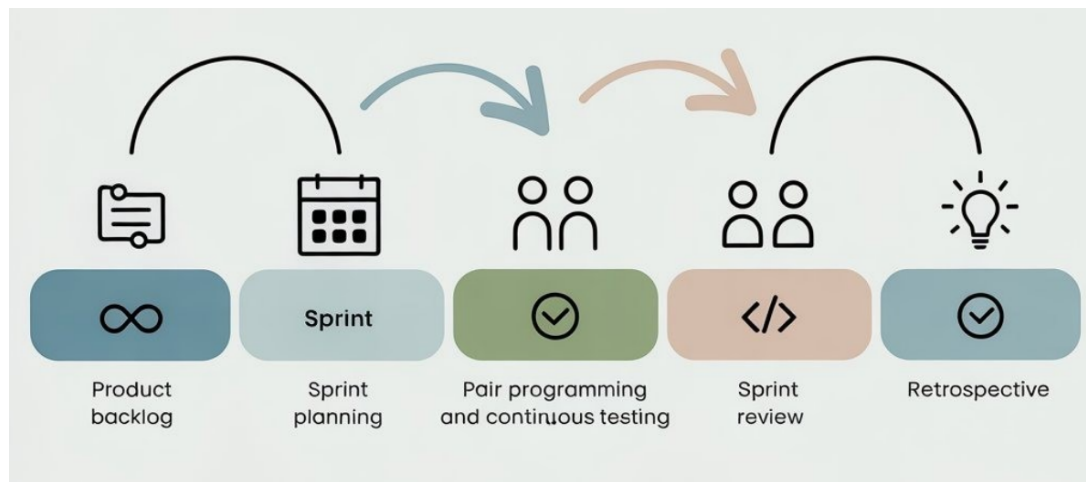
O processo de desenvolvimento será estruturado em ciclos iterativos e a organização da equipe foi definida em papéis de acordo com as necessidades do projeto, conforme descrito na seção 2.1. Durante todas as etapas, o desenvolvimento será realizado de forma colaborativa, com uso de práticas de integração contínua e validação constante, garantindo alinhamento com os princípios de melhoria contínua e qualidade do XP.

O fluxo de trabalho adotado integra a organização do Scrum com as práticas técnicas do XP, sendo composto pelas seguintes etapas: levantamento de requisitos, organização e priorização no Product Backlog, planejamento da Sprint, desenvolvimento em pares com

testes contínuos, revisão da Sprint e retrospectiva. Esse ciclo se repete ao longo do projeto, promovendo entregas incrementais e melhoria contínua do processo (Figura 2).

Dessa forma, a metodologia adotada garante alinhamento entre gestão eficiente, qualidade técnica e evolução progressiva do produto, atendendo aos objetivos estabelecidos para o projeto.

Figura 2: Fluxo de trabalho.



Fonte: elaborado pelo autor (2026).

4 DECLARAÇÃO DE ESCOPO DO PROJETO

4.1 Backlog do produto

CREDIFAB é composto por funcionalidades relacionadas aos objetivos definidos na Seção 1.3. Os itens foram organizados de acordo com seu grau de prioridade (Must, Should, Could) e distribuídos a partir da primeira sprint de implementação (Sprint 4), conforme o cronograma do projeto. Classificação utilizada:

- **Must:** Requisito essencial para o funcionamento básico do sistema.
- **Should:** Requisito importante, mas não impeditivo para o desenvolvimento inicial.
- **Could:** Requisito desejável.

Os requisitos foram obtidos através de brainstorming do grupo e análise do problema e objetivos descritos na Seção 1, também considerando testes e riscos previstos nas seções anteriores e posteriores a esta.

4.2 Perfis

A Tabela 7 lista e descreve os perfis de acesso (tipos de usuário) que existirão no produto, assim como suas permissões de acesso.

Tabela 7 – Perfis de acesso.

#	Nome do perfil	Características do perfil	Permissões de acesso
1	Gestor / Proprietário	Usuário principal da empresa, responsável por alimentar dados financeiros e tomar decisões.	Inserir receitas/despesas, cadastrar contas, anexar documentos, gerar relatórios, simular e comparar crédito, alterar dados próprios.

Fonte: Elaborada pelos autores (2026).

4.3 Cenários

A Tabela 8 descreve cada cenário que será usado como caso base para cada uma das Sprints, assim como seus objetivos individuais e Sprints relacionadas. A distribuição foi revisada nesta versão para que cada cenário ocupe um intervalo contínuo e equilibrado de Sprints.

Tabela 8 – Cenários funcionais.

ID	Nome	Sprints	Objetivos
CEN-00	Plataforma e autenticação	Sprints 4–5	Estabelecer a base da aplicação: cadastro, login, recuperação de senha, exclusão de conta e vínculo entre usuário e empresa.
CEN-01	Registro de dados financeiros	Sprints 5–7	Estruturar o fluxo de caixa da empresa por meio de categorias, transações, histórico, dashboard e contas a pagar/receber.
CEN-02	Centralização documental para crédito	Sprint 8	Organizar documentos fiscais, contábeis e jurídicos em um repositório único.
CEN-03	Geração de relatório financeiro	Sprint 8	Produzir relatórios compreensíveis (PDF) para gestores e instituições financeiras.
CEN-04	Diagnóstico e simulação de crédito	Sprint 9	Apoiar a escolha de linha de crédito, simular impacto no caixa e comparar modalidades.

Fonte: Elaborada pelos autores (2026).

4.4 Tabela de Backlog do produto

A Tabela 9 enumera, descreve e classifica os requisitos do projeto:

Tabela 9 – Backlog do produto.

ID	Nome	Prioridade	Descrição
CEN-00 / R01	Cadastro de usuário	Must	Permitir cadastro do gestor com validação de e-mail e senha.
CEN-00 / R02	Exclusão de usuário e empresa	Should	Remover conta com cascata configurada nas FKs.
CEN-00 / R03	Autenticação e login	Must	Autenticar usuário via JWT e proteger rotas.
CEN-00 / R04	Modelagem Usuário–Empresa	Must	Estabelecer o relacionamento ORM entre as entidades User e Company .
CEN-00 / R05	Recuperação de senha	Should	Permitir redefinição de senha via e-mail com token temporário.
CEN-01 / R06	Cadastro de categoria	Must	Criar categorias de receita/despesa por empresa.
CEN-01 / R07	Cadastro de transação	Must	Registrar manualmente entradas e saídas financeiras.
CEN-01 / R08	Histórico de transações	Should	Listar transações com filtros por período, tipo, categoria e valor.
CEN-01 / R09	Dashboard financeiro	Must	Consolidar saldo, receitas e despesas em indicadores visuais.
CEN-01 / R15	Cadastro de contas	Should	Registrar contas a pagar/receber com vencimento e situação.
CEN-02 / R10	Upload e organização de documentos	Must	Centralizar documentos.
CEN-03 / R11	Relatórios financeiros básicos	Must	Gerar relatórios em PDF.
CEN-04 / R12	Simulação de crédito	Should	Simular parcelas (Price/SAC) e impacto financeiro no caixa.
CEN-04 / R13	Comparação de modalidades	Should	Comparar taxas e condições de linhas de crédito.
CEN-04 / R14	Desempenho	Could	Suportar múltiplas simulações simultâneas.

Fonte: elaborado pelos autores (2026).

5 MÉTRICAS E MEDIÇÕES

5.1 GQM de medições

A definição das métricas seguiu o método Goal–Question–Metric (GQM), priorizando indicadores simples, de coleta direta nas ferramentas já utilizadas pela equipe (GitHub e pipelines de CI/CD) e relacionados às práticas de Scrum e XP adotadas no projeto.

Tabela 10 – GQM Medições.

Objetivo (Goal)	Questão (Question)	Métrica (Metric)
G1: Acompanhar a capacidade de entrega da equipe ao longo das Sprints.	Q1: Quantas tarefas a equipe é capaz de concluir por Sprint?	M1: Throughput por Sprint.
G2: Avaliar a qualidade técnica do código entregue.	Q2: Qual a proporção de defeitos no software entregue?	M2: Densidade de Defeitos.
G2: Avaliar a qualidade técnica do código entregue.	Q3: O código está adequadamente coberto por testes automatizados?	M3: Cobertura de Testes.
G3: Avaliar a estabilidade da integração contínua.	Q4: Os Pull Requests estão sendo submetidos com qualidade suficiente para passar na pipeline?	M4: Taxa de Aprovação da Pipeline.

Fonte: elaborado pelo autor (2026).

Detalhamento da Métrica 1: Throughput por Sprint

Definição da Métrica: Quantidade de issues (tarefas ou histórias de usuário) concluídas pela equipe ao final de cada Sprint.

5W1H:

- **What (O quê):** Número de issues fechadas com status *Done* ao final da Sprint.
- **Why (Por Quê):** Estimar com mais precisão a carga de trabalho viável para as próximas Sprints, evitando ociosidade ou sobrecarga da equipe.
- **Who (Quem):** Dupla de Qualidade, com apoio da equipe de desenvolvimento.
- **Where (Onde):** GitHub Projects (board do repositório do projeto).
- **When (Quando):** Ao final de cada Sprint, durante a reunião de revisão.
- **How (Como):** Contagem das issues movidas para a coluna *Done* durante a Sprint. Issues parcialmente concluídas ou ainda em revisão não são contabilizadas.

Informações Adicionais:

- **Forma de cálculo:** Soma do número de issues concluídas na Sprint.
- **Unidade:** issues / Sprint.
- **Valores esperados:** Oscilação nas primeiras Sprints, com tendência de estabilização a partir da terceira.
- **Forma de análise:** Os valores são registrados em uma planilha simples para acompanhamento da evolução. Caso o throughput caia mais de 30% em relação à média das Sprints anteriores, o tema é discutido na retrospectiva, junto com as possíveis causas (escopo subestimado, ausência de integrantes, dívida técnica, entre outras).

Detalhamento da Métrica 2: Densidade de Defeitos

Definição da Métrica: Relação entre o número de defeitos identificados pela dupla de Qualidade e o volume de tarefas concluídas na Sprint.

5W1H:

- **What (O quê):** Defeitos validados pela dupla de Qualidade durante a Sprint.
- **Why (Por Quê):** Identificar oscilações na qualidade do código entregue e orientar discussões na retrospectiva.
- **Who (Quem):** Dupla de Qualidade.
- **Where (Onde):** GitHub Issues do repositório, identificadas com a label **bug**.
- **When (Quando):** Ao final de cada Sprint.
- **How (Como):** Total de issues com label **bug** abertas durante a Sprint, dividido pelo total de issues concluídas na mesma Sprint.

Informações Adicionais:

- **Forma de cálculo:** $D = \frac{\text{nº de bugs registrados}}{\text{nº de issues concluídas}}$
- **Unidade:** bugs / issue concluída.
- **Valores esperados:** O valor ideal é o mais próximo de zero possível. Espera-se uma redução gradual ao longo das Sprints, conforme a equipe amadurece nas práticas de teste e pareamento.
- **Forma de análise:** Se o valor aumentar em duas Sprints consecutivas, a equipe discute na retrospectiva as causas prováveis (complexidade técnica, ausência de testes, pareamento ineficiente) e propõe ajustes nas práticas de desenvolvimento.

Detalhamento da Métrica 3: Cobertura de Testes

Definição da Métrica: Percentual de linhas do código-fonte da aplicação que são executadas pelos testes automatizados.

5W1H:

- **What (O quê):** Cobertura de linha (*line coverage*) do backend e do frontend.
- **Why (Por Quê):** Verificar se a evolução do código está acompanhada da escrita de testes correspondentes, reduzindo o risco de regressões em Sprints futuras.
- **Who (Quem):** Dupla de Qualidade.
- **Where (Onde):** Relatórios gerados pelo `pytest-cov` (backend) e pelo `vitest-coverage` (frontend), disponíveis nos logs da pipeline do GitHub Actions.
- **When (Quando):** A cada Pull Request para a branch `develop` e consolidação ao final da Sprint.
- **How (Como):** Extraíndo o percentual de cobertura reportado pelas ferramentas e calculando a média entre backend e frontend.

Informações Adicionais:

- **Forma de cálculo:** $C = \frac{C_{backend} + C_{frontend}}{2}$
- **Unidade:** %.
- **Valores esperados:** Mínimo aceitável de 60% nas Sprints iniciais (4–5), com evolução prevista até 75% nas Sprints finais (6–10). Os limites podem ser revisados pela equipe conforme a complexidade dos módulos em desenvolvimento.
- **Forma de análise:** Se a cobertura cair entre duas Sprints consecutivas, ou ficar abaixo do mínimo aceitável, a dupla de Qualidade sinaliza o problema na retrospectiva e prioriza escrita de testes nas Sprints seguintes.

Detalhamento da Métrica 4: Taxa de Aprovação da Pipeline

Definição da Métrica: Percentual de Pull Requests que passam na pipeline do GitHub Actions na primeira execução, sem necessidade de novos commits para correção.

5W1H:

- **What (O quê):** Quantidade de PRs aprovados na primeira execução da pipeline, em relação ao total de PRs abertos na Sprint.
- **Why (Por Quê):** Avaliar indiretamente a maturidade das práticas de XP, refletindo a qualidade do código submetido antes da revisão humana.
- **Who (Quem):** Dupla de Qualidade.

- **Where (Onde):** Histórico de execuções do GitHub Actions e lista de Pull Requests do repositório.
- **When (Quando):** Ao final de cada Sprint.
- **How (Como):** Contagem dos PRs cujo primeiro *run* da pipeline foi aprovado, dividida pelo total de PRs daquela Sprint.

Informações Adicionais:

- **Forma de cálculo:** $T = \frac{\text{n}^\circ \text{ de PRs aprovados no 1}^\circ \text{ run}}{\text{n}^\circ \text{ total de PRs da Sprint}} \times 100$
- **Unidade:** %.
- **Valores esperados:** Taxa baixa nas primeiras Sprints, com evolução gradual conforme a equipe amadurece. Meta de superar 70% a partir da quinta Sprint.
- **Forma de análise:** Se a taxa estiver baixa ou em queda, a dupla de Qualidade investiga as causas mais frequentes das falhas (testes esquecidos, falhas de lint, ausência de execução local antes do commit) e propõe ações de correção na retrospectiva.

5.2 Consolidação das Métricas por Sprint

A Tabela 11 consolida os valores coletados ao longo das Sprints de construção. Os valores das Sprints 4 a 8 são realizados; os das Sprints 9 e 10 são projetados a partir da tendência observada e das pendências já mapeadas.

Tabela 11 – Consolidação GQM por Sprint.

Sprint	M1 (issues)	M2 (bugs/issue)	M3 (cobertura)	M4 (pipeline)
Sprint 4	5	0,40	63%	– (CI em implantação)
Sprint 5	6	0,33	69%	67%
Sprint 6	6	0,33	73%	72%
Sprint 7	7	0,29	77%	79%
Sprint 8	6	0,33	80%	83%
Sprint 9*				
Sprint 10*				

*Fonte: elaborado pelos autores (2026). *Valores projetados.*

A leitura agregada mostra throughput estável (média de 6 issues/Sprint), densidade de defeitos em queda à medida que as práticas de teste amadurecem, cobertura ultrapassando o limite de 75% a partir da Sprint 6 e taxa de aprovação da pipeline superando a meta de 70% a partir da Sprint 5.

6 TESTES DE SOFTWARE

6.1 Estratégia de Testes

A estratégia de testes adotada busca garantir o funcionamento correto das regras de negócio e a estabilidade da aplicação nas camadas de Frontend, Backend e Banco de Dados. A estratégia prioriza simplicidade, rastreabilidade e transparência sobre o que foi efetivamente testado em cada feature, seguindo a ideia de pirâmide de testes: muitos testes unitários, menos de integração, menos ainda de sistema (E2E) e testes de carga sob demanda.

Níveis de Testes Abordados

- **Unitário:** Testes isolados de funções e componentes. No backend, foco na precisão dos cálculos das regras de negócio e nas validações, utilizando Pytest e pytest-mock. No frontend, foco na renderização de componentes UI isolados e em hooks de estado, utilizando Vitest e React Testing Library.
- **Integração:** Testes da comunicação entre módulos, com foco na interação das rotas da API com o banco de dados (SQLite em memória) e na integração dos componentes do frontend com os endpoints do backend.
- **Sistema (E2E):** Testes que simulam o fluxo completo do usuário, da interface até a resposta do servidor, utilizando Playwright.

Tipos de Testes Abordados

O foco principal está em **testes funcionais**: validação das regras de negócio (cálculos financeiros e simulação de crédito), fluxos de autenticação e persistência correta dos dados no banco.

De forma pontual, a equipe executa um **teste de carga simples com Locust** próximo ao final do projeto, restrito ao endpoint de simulação de crédito, para verificar se a hospedagem gratuita do Render suporta o uso esperado durante a apresentação final. Não há compromisso com monitoramento contínuo de desempenho ao longo das Sprints.

A medição de cobertura (M3) é feita com `pytest-cov` no backend e `vitest -coverage` no frontend. A construção de dados de teste é feita com fixtures do Pytest e mocks via `pytest-mock`, mantendo a suíte simples e sem dependências adicionais.

Fluxo de Trabalho dos Testes

O processo de testes acompanha o Git Flow do projeto e é executado para cada feature concluída pelo time de desenvolvimento, conforme o ciclo a seguir:

1. A equipe de desenvolvimento finaliza a feature em sua branch **feature/<nome>**.
2. A dupla de Qualidade cria uma branch **test/<nome-da-feature>** a partir da branch da feature.
3. Na branch de teste, são escritos e executados os scripts (unitários, integração e, quando aplicável, E2E).
4. Os resultados são documentados conforme descrito em *Documentação dos Testes*.
5. Se todos os testes passam: a branch **test/** é apagada e abre-se um Pull Request da branch de feature para **develop**.
6. Se algum teste falha: é criada uma branch **fix/<nome-do-defeito>** a partir da branch da feature, a correção é desenvolvida pelo time, e o ciclo de teste recomeça do passo 2.

Esse fluxo garante que a branch **develop** só receba código que passou pelo ciclo completo de testes da dupla de Qualidade.

Documentação dos Testes

Para cada feature testada, a dupla de Qualidade produz um documento próprio (publicado no MkDocs) contendo:

- identificação da feature e da branch testada;
- lista dos casos de teste executados (referenciando o Roteiro de Testes);
- resultado esperado e resultado observado para cada caso;
- evidências (prints, logs ou trechos de traceback, quando aplicáveis);
- defeitos encontrados, com referência à issue correspondente no GitHub;
- status final da feature: aprovada, reprovada ou aprovada com pendências.

Esses documentos compõem o histórico de qualidade do projeto e permitem rastrear, em qualquer momento da disciplina, o que foi efetivamente testado em cada entrega.

Ambientes de Teste

- **Ambiente Local:** Ambiente da dupla de Qualidade, utilizado para criação e execução dos scripts de teste na branch `test/`.
- **Ambiente de Integração Contínua (CI):** Ao abrir um Pull Request da branch de feature para a `develop`, o GitHub Actions é acionado automaticamente e executa os testes unitários e de integração (backend e frontend), além de coletar a cobertura. O merge só é autorizado se a pipeline for aprovada.

Forma de Análise dos Testes

Os testes são avaliados a partir da saída dos scripts e dos logs do GitHub Actions. Em caso de falha, a dupla de Qualidade analisa o traceback junto ao desenvolvedor responsável, registra o defeito como issue com a label `status: Com Pendências` no repositório, e acompanha a correção até a nova aprovação dos testes na branch `fix/` correspondente.

Resultados Obtidos (Previstos x Realizados)

Cada caso de teste possui um resultado esperado descrito no Roteiro de Testes. Qualquer divergência entre o esperado e o comportamento real da aplicação é classificada como defeito, documentada no relatório da feature, corrigida na branch `fix/` correspondente, e o ciclo de teste é repetido até que o realizado corresponda ao previsto, atingindo o critério de aprovação.

Até a Sprint 8, as features dos cenários CEN-00 a CEN-03 foram testadas e validadas, com cobertura de núcleo de negócio acima da meta. Os defeitos identificados (por exemplo, validação de senha com espaços, cálculo de totais com filtro envolvendo `Decimal/float`, e registro de blueprint/relacionamentos do módulo de crédito) foram registrados como issues `bug` e encaminhados para correção em branches `fix/`, sendo consolidados na Sprint de estabilização. Os testes do cenário CEN-04 (simulação e comparação de crédito) tiveram a lógica de negócio totalmente coberta no nível unitário e estão em fechamento na Sprint 9, após a correção dos defeitos de persistência.

6.2 Roteiro de Teste

A tabela abaixo apresenta uma visão resumida dos casos de teste planejados para o projeto, atualizada nesta versão para refletir os níveis efetivamente cobertos (unitário, integração, sistema e carga) e os requisitos adicionados.

Tabela 12 – Roteiro de Testes.

Cód.	Nome do Teste	Nível	Tipo	Req.
TS-01	Validação de e-mail e senha no cadastro de usuário	Unitário	Funcional	R01
TS-02	Validação de data de nascimento e idade mínima	Unitário	Funcional	R01
TS-03	Geração e validação de token JWT	Unitário	Funcional	R03
TS-04	Renderização dos formulários de cadastro e login	Unitário	Funcional	R01, R03
TS-05	Validação de transação (valor, tipo, categoria)	Unitário	Funcional	R07
TS-06	Cálculo de financiamento pela Tabela Price	Unitário	Funcional	R12
TS-07	Cálculo de financiamento pelo SAC	Unitário	Funcional	R12
TS-08	Projeção de impacto no fluxo de caixa	Unitário	Funcional	R12
TS-09	Cálculo e destaque da modalidade mais vantajosa	Unitário	Funcional	R13
TS-10	Alerta de modalidade de pessoa física	Unitário	Funcional	R13
TS-11	Filtros e paginação do histórico (hook de UI)	Unitário	Funcional	R08
TS-12	Autenticação e login via token JWT	Integração	Funcional	R03
TS-13	Recuperação de senha via token temporário	Integração	Funcional	R05
TS-14	Exclusão de usuário/empresa com cascata nas FKs	Integração	Funcional	R02
TS-15	Cadastro de categoria por empresa (unicidade)	Integração	Funcional	R06

Cód.	Nome do Teste	Nível	Tipo	Req.
TS-16	Cadastro de transação vinculada à categoria via API	Integração	Funcional	R07
TS-17	Histórico de transações com filtros e totais	Integração	Funcional	R08
TS-18	Isolamento de dados entre empresas (vínculo usuário–empresa)	Integração	Funcional	R04
TS-19	Upload e organização de documentos	Integração	Funcional	R10
TS-20	Cadastro de contas a pagar/receber	Integração	Funcional	R15
TS-21	Geração de relatório financeiro em PDF	Integração	Funcional	R11
TS-22	Endpoints de simulação de crédito (calcular/salvar/listar/excluir)	Integração	Funcional	R12
TS-23	Endpoints de comparação de modalidades	Integração	Funcional	R13
TS-24	Fluxo E2E de cadastro, login e registro de transação	Sistema	Funcional	R01, R03, R07
TS-25	Fluxo E2E de consulta ao histórico financeiro com filtros	Sistema	Funcional	R08
TS-26	Fluxo E2E de simulação de crédito e visualização do impacto	Sistema	Funcional	R12
TS-27	Teste de carga no endpoint de simulação de crédito (Locust)	Carga	Não funcional	R14

Fonte: Elaborado pelo autor (2026).

A matriz completa, contendo pré-condições, passos de execução, dados de entrada, resultado esperado, status e demais colunas de verificação, é mantida em formato de planilha eletrônica para facilitar a visualização e a manutenção contínua, e replicada de forma navegável na documentação do projeto (MkDocs).

A matriz de testes completa está disponível em: <https://docs.google.com/spreadsheets/d/1I7Xj4GSMts4-ospICI8isJ7PIg6oiPaZCJNpZC190zQ/edit?usp=sharing>.

7 REFERÊNCIAS BIBLIOGRÁFICAS

- SEBRAE. **Relatório de Inteligência: Barreiras no Acesso ao Crédito**. Curitiba: SEBRAE/PR, jun. 2025. Disponível em: <https://sebraepr.com.br/comunidade/artigo/relatorio-de-inteligencia-barreiras-no-acesso-ao-credito>.
- BRASIL. Ministério da Economia; ESCOLA NACIONAL DE ADMINISTRAÇÃO PÚBLICA (ENAP). **Desafios de Acesso a Crédito – Briefing dos 3 problemas**. Programa Catálise. Brasília, DF: Ministério da Economia, [s.d.]
- BUENO, V. F. F. **Avaliação de risco na concessão de crédito bancário para micros e pequenas empresas**. 2003. Dissertação (Mestrado em Engenharia de Produção) – Programa de Pós-Graduação em Engenharia de Produção, Universidade Federal de Santa Catarina, Florianópolis, 2003.
- GUIMARÃES, L. **MPEs têm dificuldade de acesso a crédito, mas entraves podem estar na gestão**. CNN Brasil, [S. l.], 28 jul. 2021.
- PIRES, W.; TERENCE, R. C. **A concessão de crédito para as micro e pequenas empresas**. Revista Ciência Contemporânea, [S. l.], v. 2, n. 1, p. 225-237, jun./dez. 2017. Disponível em: http://uniesp.edu.br/sites/guaratingueta/revista.php?id_revista=31.
- PREISLER, A. M. **Análise de risco e crédito para micro e pequenas empresas: uma proposta orientativa**. 2003. 180 f. Dissertação (Mestrado em Engenharia de Produção) – Programa de Pós-Graduação em Engenharia de Produção, Universidade Federal de Santa Catarina, Florianópolis, 2003.